📚 Complete SQL with Python Manual

> **Welcome!** This comprehensive guide will teach you everything you need to know about working with SQLite databases in Python. By the end of this manual, you'll be able to create, read, update, and delete data with confidence.
> **Prerequisites:** Basic Python knowledge and basic SQL understanding

📋 Table of Contents

1. Introduction to SQLite3

SQLite3 is a lightweight, serverless database that comes built-in with Python. It's perfect for small to medium-sized applications, prototyping, and learning database concepts.

> **Why SQLite3?**
>
> - No separate server required - the database is just a file
> - Built into Python's standard library - no installation needed
> - Fast and reliable for most applications
> - Great for learning SQL and database concepts

2. Connecting to Database

The first step in working with any database is establishing a connection. Here's how to do it with SQLite3:

2.1 Import the Module

```
import sqlite3
```

2.2 Create a Connection

```
# Connect to database (creates file if it doesn't exist)
connection = sqlite3.connect('my_database.db')

# For in-memory database (temporary, deleted when program ends)
connection = sqlite3.connect(':memory:')
```

2.3 Complete Connection Example

```
import sqlite3

# Create connection
connection = sqlite3.connect('company.db')

# Create cursor (we'll explain this next)
cursor = connection.cursor()

print("Successfully connected to database!")

# Always close connection when done
connection.close()
```

Key Points:

- `connect()` creates a database file if it doesn't exist
- The database file is created in your current directory
- Always close the connection when you're done: `connection.close()`

3. Understanding Cursor Objects

The cursor is your primary tool for executing SQL commands. Think of it as a "pointer" that lets you navigate and manipulate the database.

3.1 Creating a Cursor

```
cursor = connection.cursor()
```

3.2 What Does the Cursor Do?

- **Execute SQL commands:** `cursor.execute(sql_command)`
- **Fetch results:** `cursor.fetchall()` , `cursor.fetchone()`
- **Navigate through results:** Move through query results row by row

> **Important:** After executing commands that modify data (INSERT, UPDATE, DELETE), you must call `connection.commit()` to save changes!

4. Creating Tables

Tables are where your data lives. Let's learn how to create them.

4.1 Basic Table Creation

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# SQL command to create table
sql_command = """
CREATE TABLE employees (
    id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    age INTEGER,
    department TEXT,
    salary REAL
)
"""

# Execute the command
cursor.execute(sql_command)

print("Table created successfully!")

connection.close()
```

4.2 Common Data Types

SQLite Type	Description	Example
INTEGER	Whole numbers	1, 42, -10
REAL	Decimal numbers	3.14, 50000.50
TEXT	Strings of text	"John", "Hello World"
BLOB	Binary data	Images, files
NULL	No value	NULL

4.3 Key Constraints

- `PRIMARY KEY` - Unique identifier for each row
- `NOT NULL` - Field cannot be empty
- `UNIQUE` - All values must be different
- `DEFAULT` - Set a default value

5. Inserting Data (CREATE)

Now let's add data to your table!

5.1 Insert Single Row

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# Insert one record
sql_command = """
INSERT INTO employees (id, first_name, last_name, age, department, sal
VALUES (1, 'John', 'Smith', 30, 'Sales', 50000.00)
"""

cursor.execute(sql_command)

# IMPORTANT: Save changes!
connection.commit()

print("Record inserted successfully!")

connection.close()
```

5.2 Insert Multiple Rows

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# Data to insert
employees = [
    (2, 'Sarah', 'Johnson', 28, 'Engineering', 75000.00),
    (3, 'Mike', 'Williams', 35, 'Marketing', 60000.00),
    (4, 'Emily', 'Brown', 32, 'HR', 55000.00)
]

# Insert multiple records
cursor.executemany("""
    INSERT INTO employees VALUES (?, ?, ?, ?, ?, ?)
""", employees)

connection.commit()

print(f"Inserted {len(employees)} records!")

connection.close()
```

5.3 Safe Insert with User Input (Preventing SQL Injection)

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# User input
first_name = input("First name: ")
last_name = input("Last name: ")
age = int(input("Age: "))
department = input("Department: ")
salary = float(input("Salary: "))

# Safe way using placeholders (?)
cursor.execute("""
    INSERT INTO employees (first_name, last_name, age, department, sal
    VALUES (?, ?, ?, ?, ?)
""", (first_name, last_name, age, department, salary))

connection.commit()
connection.close()
```

> **Why use ? placeholders?** They protect against SQL injection attacks and automatically handle data types and escaping special characters.

6. Reading Data (READ)

Let's learn how to retrieve data from your database.

6.1 Fetch All Records

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# Select all records
cursor.execute("SELECT * FROM employees")

# Fetch all results
results = cursor.fetchall()

# Display results
for row in results:
    print(row)

# Output example:
# (1, 'John', 'Smith', 30, 'Sales', 50000.0)
# (2, 'Sarah', 'Johnson', 28, 'Engineering', 75000.0)

connection.close()
```

6.2 Fetch One Record

```
cursor.execute("SELECT * FROM employees")

# Fetch one record at a time
row = cursor.fetchone()
print(row)  # First row

row = cursor.fetchone()
print(row)  # Second row
```

6.3 Fetch Limited Number of Records

```
cursor.execute("SELECT * FROM employees")

# Fetch first 3 records
results = cursor.fetchmany(3)

for row in results:
    print(row)
```

6.4 Select Specific Columns

```
# Select only name and salary
cursor.execute("SELECT first_name, last_name, salary FROM employees")

results = cursor.fetchall()

for row in results:
    print(f"{row[0]} {row[1]}: ${row[2]}")

# Output:
# John Smith: $50000.0
# Sarah Johnson: $75000.0
```

6.5 Fetch Methods Comparison

Method	Returns	Use Case
fetchall()	List of all rows	When you need all results
fetchone()	Single row (or None)	Process results one by one
fetchmany(n)	List of n rows	Get a specific number of results

7. Updating Data (UPDATE)

Modify existing records in your database.

7.1 Update Single Record

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# Update John's salary
cursor.execute("""
    UPDATE employees
    SET salary = 55000.00
    WHERE first_name = 'John' AND last_name = 'Smith'
""")

connection.commit()

print(f"Updated {cursor.rowcount} record(s)")

connection.close()
```

7.2 Update Multiple Fields

```
# Update multiple fields at once
cursor.execute("""
    UPDATE employees
    SET age = 31, department = 'Management', salary = 65000.00
    WHERE id = 1
""")

connection.commit()
```

7.3 Safe Update with Parameters

```
# User input
employee_id = 2
new_salary = 80000.00

# Safe update with placeholders
cursor.execute("""
    UPDATE employees
    SET salary = ?
    WHERE id = ?
""", (new_salary, employee_id))

connection.commit()
```

7.4 Update All Records (Be Careful!)

```
# Give everyone a 10% raise
cursor.execute("""
    UPDATE employees
    SET salary = salary * 1.10
""")

connection.commit()
```

> **Warning:** Without a WHERE clause, UPDATE affects ALL records! Always double-check your WHERE conditions.

8. Deleting Data (DELETE)

Remove records from your database.

8.1 Delete Specific Record

```
import sqlite3

connection = sqlite3.connect('company.db')
cursor = connection.cursor()

# Delete by ID
cursor.execute("""
    DELETE FROM employees
    WHERE id = 1
""")

connection.commit()

print(f"Deleted {cursor.rowcount} record(s)")

connection.close()
```

8.2 Delete Multiple Records

```
# Delete all employees in a department
cursor.execute("""
    DELETE FROM employees
    WHERE department = 'Sales'
""")

connection.commit()
```

8.3 Delete with Conditions

```
# Delete employees with salary over 70000
cursor.execute("""
    DELETE FROM employees
    WHERE salary > 70000
""")

connection.commit()
```

8.4 Delete All Records

```
# WARNING: This deletes EVERYTHING!
cursor.execute("DELETE FROM employees")
connection.commit()
```

8.5 Drop Entire Table

```
# Completely remove the table
cursor.execute("DROP TABLE employees")
connection.commit()
```

> **Critical:** DELETE removes data permanently. DROP removes the entire table structure. Always have backups!

9. Searching & Filtering Data

Learn how to find specific data using various conditions.

9.1 WHERE Clause - Basic Filtering

```
# Find employees in Engineering
cursor.execute("""
    SELECT * FROM employees
    WHERE department = 'Engineering'
""")

results = cursor.fetchall()
```

9.2 Comparison Operators

```
# Salary greater than 60000
cursor.execute("SELECT * FROM employees WHERE salary > 60000")

# Age less than or equal to 30
cursor.execute("SELECT * FROM employees WHERE age <= 30")

# Not equal to Sales
cursor.execute("SELECT * FROM employees WHERE department != 'Sales'")
```

9.3 AND / OR / NOT Operators

```
# AND - Both conditions must be true
cursor.execute("""
    SELECT * FROM employees
    WHERE age > 30 AND salary > 60000
""")

# OR - Either condition can be true
cursor.execute("""
    SELECT * FROM employees
    WHERE department = 'Sales' OR department = 'Marketing'
""")

# NOT - Exclude condition
cursor.execute("""
    SELECT * FROM employees
    WHERE NOT department = 'HR'
""")
```

9.4 BETWEEN - Range Filtering

```
# Salary between 50000 and 70000
cursor.execute("""
    SELECT * FROM employees
    WHERE salary BETWEEN 50000 AND 70000
""")

# Age between 25 and 35
cursor.execute("""
    SELECT * FROM employees
    WHERE age BETWEEN 25 AND 35
""")
```

9.5 IN - Multiple Values

```
# Find employees in specific departments
cursor.execute("""
    SELECT * FROM employees
    WHERE department IN ('Sales', 'Marketing', 'HR')
""")
```

9.6 LIKE - Pattern Matching

```
# Names starting with 'S'
cursor.execute("""
    SELECT * FROM employees
    WHERE first_name LIKE 'S%'
""")

# Names ending with 'son'
cursor.execute("""
    SELECT * FROM employees
    WHERE last_name LIKE '%son'
""")

# Names containing 'oh'
cursor.execute("""
    SELECT * FROM employees
    WHERE first_name LIKE '%oh%'
""")
```

9.7 ORDER BY - Sorting

```
# Sort by salary (ascending - low to high)
cursor.execute("""
    SELECT * FROM employees
    ORDER BY salary ASC
""")

# Sort by salary (descending - high to low)
cursor.execute("""
    SELECT * FROM employees
    ORDER BY salary DESC
""")

# Sort by multiple columns
cursor.execute("""
    SELECT * FROM employees
    ORDER BY department ASC, salary DESC
""")
```

9.8 LIMIT - Restrict Number of Results

```
# Get top 3 highest paid employees
cursor.execute("""
    SELECT * FROM employees
    ORDER BY salary DESC
    LIMIT 3
""")

# Get employees 5-10 (skip first 5)
cursor.execute("""
    SELECT * FROM employees
    LIMIT 5 OFFSET 5
""")
```

9.9 COUNT, AVG, SUM, MIN, MAX - Aggregate Functions

```
# Count total employees
cursor.execute("SELECT COUNT(*) FROM employees")
print(f"Total employees: {cursor.fetchone()[0]}")

# Average salary
cursor.execute("SELECT AVG(salary) FROM employees")
print(f"Average salary: {cursor.fetchone()[0]}")

# Total salary expense
cursor.execute("SELECT SUM(salary) FROM employees")

# Highest salary
cursor.execute("SELECT MAX(salary) FROM employees")

# Lowest salary
cursor.execute("SELECT MIN(salary) FROM employees")
```

9.10 GROUP BY - Grouping Results

```
# Count employees per department
cursor.execute("""
    SELECT department, COUNT(*) as employee_count
    FROM employees
    GROUP BY department
""")

# Average salary per department
cursor.execute("""
    SELECT department, AVG(salary) as avg_salary
    FROM employees
    GROUP BY department
""")
```

10. Advanced Operations

10.1 Complete CRUD Example

```
import sqlite3

def create_connection(db_file):
    """Create a database connection"""
    try:
        conn = sqlite3.connect(db_file)
        return conn
    except Error as e:
        print(e)
    return None

def create_table(conn):
    """Create employees table"""
    sql = """
    CREATE TABLE IF NOT EXISTS employees (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        first_name TEXT NOT NULL,
        last_name TEXT NOT NULL,
        age INTEGER,
        department TEXT,
        salary REAL
    )
    """
    conn.execute(sql)
    conn.commit()

def insert_employee(conn, employee):
    """Insert a new employee"""
    sql = """
    INSERT INTO employees (first_name, last_name, age, department, sal
    VALUES (?, ?, ?, ?, ?)
    """
    cursor = conn.cursor()
```

```
    cursor.execute(sql, employee)
    conn.commit()
    return cursor.lastrowid

def get_all_employees(conn):
    """Retrieve all employees"""
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM employees")
    return cursor.fetchall()

def update_employee_salary(conn, employee_id, new_salary):
    """Update employee salary"""
    sql = "UPDATE employees SET salary = ? WHERE id = ?"
    conn.execute(sql, (new_salary, employee_id))
    conn.commit()

def delete_employee(conn, employee_id):
    """Delete an employee"""
    sql = "DELETE FROM employees WHERE id = ?"
    conn.execute(sql, (employee_id,))
    conn.commit()

# Main program
if __name__ == '__main__':
    conn = create_connection('company.db')

    # Create table
    create_table(conn)

    # Insert employee
    employee = ('John', 'Doe', 30, 'IT', 60000)
    employee_id = insert_employee(conn, employee)
    print(f"Inserted employee with ID: {employee_id}")

    # Get all employees
    employees = get_all_employees(conn)
    for emp in employees:
        print(emp)
```

```
    # Update salary
    update_employee_salary(conn, employee_id, 65000)

    # Delete employee
    # delete_employee(conn, employee_id)

    conn.close()
```

10.2 Using Context Manager (Best Practice)

```
import sqlite3

# Context manager automatically closes connection
with sqlite3.connect('company.db') as conn:
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM employees")
    results = cursor.fetchall()

    for row in results:
        print(row)

# Connection automatically closed here!
```

10.3 Error Handling

```
import sqlite3

try:
    conn = sqlite3.connect('company.db')
    cursor = conn.cursor()

    cursor.execute("""
        INSERT INTO employees (first_name, last_name, age, department,
        VALUES (?, ?, ?, ?, ?)
    """, ('Jane', 'Smith', 28, 'Sales', 55000))

    conn.commit()
    print("Employee added successfully!")

except sqlite3.IntegrityError:
    print("Error: Duplicate entry or constraint violation")
except sqlite3.Error as e:
    print(f"Database error: {e}")
finally:
    if conn:
        conn.close()
```

11. Best Practices

✓ Do's:

- **Always use placeholders (?) for user input** to prevent SQL injection
- **Call commit()** after INSERT, UPDATE, or DELETE operations
- **Close connections** when done or use context managers
- **Use try-except blocks** for error handling

- **Use AUTOINCREMENT** for primary keys
- **Add indexes** on frequently searched columns for better performance

✗ Don'ts:

- **Never concatenate user input** directly into SQL strings
- **Don't forget WHERE clauses** in UPDATE/DELETE (unless intentional)
- **Don't store sensitive data** without encryption
- **Avoid SELECT *** in production - specify columns you need

11.1 SQL Injection Example (BAD)

```
# NEVER DO THIS!
user_input = input("Enter name: ")
cursor.execute(f"SELECT * FROM employees WHERE first_name = '{user_inp
# If user enters: John' OR '1'='1
# Query becomes: SELECT * FROM employees WHERE first_name = 'John' OR
# This returns ALL records!
```

11.2 SQL Injection Prevention (GOOD)

```
# ALWAYS USE THIS!
user_input = input("Enter name: ")
cursor.execute("SELECT * FROM employees WHERE first_name = ?", (user_i
# The placeholder safely escapes the input
```

11.3 Quick Reference Table

Operation	SQL Command	Python Method
Create Table	CREATE TABLE	execute()
Insert Data	INSERT INTO	execute() / executemany()
Read Data	SELECT	execute() + fetchall()
Update Data	UPDATE	execute() + commit()
Delete Data	DELETE	execute() + commit()
Drop Table	DROP TABLE	execute() + commit()

🎉 Congratulations!

You now have complete knowledge of working with SQLite databases in Python! You've learned:

- ✓ How to connect to databases
- ✓ All CRUD operations (Create, Read, Update, Delete)
- ✓ Searching and filtering data
- ✓ Advanced queries with aggregate functions
- ✓ Best practices and security considerations

> **Practice Makes Perfect!** Try building a simple project like:
>
> - 📝 To-do list manager

- 📚 Book collection tracker
- 💰 Personal expense tracker
- 👥 Contact management system

Happy Coding! 🐍

Print this manual to PDF using your browser's Print function (Ctrl+P / Cmd+P)